Sudoku as a Constraint Satisfaction Problem

Introduction

Sudoku is a widely known logic puzzle that can be naturally modelled as a constraint satisfaction problem (CSP). In a standard 9×9 Sudoku grid, the objective is to assign digits from 1 to 9 to each cell such that no digit repeats within any row, column, or 3×3 sub-grid. This structure makes Sudoku a suitable domain for studying fundamental artificial intelligence techniques, particularly search algorithms, heuristics, and constraint propagation.

Constraint satisfaction problems are defined by a set of variables, domains for each variable, and constraints that restrict allowable assignments [3, 5]. In the case of Sudoku, each cell corresponds to a variable, the domain consists of digits 1 through 9, and the constraints enforce the uniqueness rules of the puzzle. Solving Sudoku therefore involves finding a complete assignment of values to variables that satisfies all constraints simultaneously. As Kumar [3] noted in his survey of CSP algorithms, a wide range of problems in artificial intelligence and computer science can be formulated as constraint satisfaction problems, including machine vision, scheduling, temporal reasoning, and graph colouring. The two primary approaches to solving CSPs are constraint propagation, which simplifies the problem by eliminating inconsistent values, and backtracking search, which systematically explores the space of possible assignments. Most effective solvers combine both approaches.

The primary challenge in solving Sudoku lies in the size of the search space. A naive approach that tries every possible combination of digits would need to explore up to 9^{81} assignments in the worst case, which is computationally infeasible. As a result, efficient solving requires the use of heuristics and constraint propagation techniques to prune the search tree and avoid unnecessary exploration [5].

The foundational techniques for solving CSPs efficiently were established in several key papers. Mackworth [4] laid the theoretical groundwork for constraint propagation with his paper *Consistency in Networks of Relations*. Mackworth formalized the concepts of node consistency, arc consistency, and path consistency as properties of constraint networks, and introduced the AC-3 algorithm for enforcing arc consistency. Arc consistency is a stronger form of constraint propagation than forward checking: while forward checking only removes values from the

domains of a variable's immediate neighbours after an assignment, arc consistency iteratively ensures that for every value in every variable's domain, there exists a consistent value in every neighbouring variable's domain, even before any assignment is made. Mackworth's work demonstrated that enforcing local consistency properties before or during search can dramatically reduce the number of backtracks needed to find a solution. The AC-3 algorithm and its successors remain widely used in modern CSP solvers [1] and represent a natural extension beyond the forward checking approach used in this project.

Building on Mackworth's consistency framework, Haralick and Elliott [2] conducted one of the earliest systematic analyses of constraint satisfaction search strategies. Their paper, *Increasing Tree Search Efficiency for Constraint Satisfaction Problems*, introduced forward checking as a lookahead technique and demonstrated that it substantially reduces the number of nodes explored during search compared to naive backtracking. In their experiments, forward checking was applied after each variable assignment to propagate constraints to unassigned neighbours; if any neighbour's domain was reduced to the empty set, the algorithm immediately backtracked without exploring the subtree further. Haralick and Elliott also formalized the role of variable ordering heuristics, showing that selecting the most constrained variable first leads to earlier detection of dead ends and more efficient pruning. Their experimental results across several CSP domains, including graph colouring and crossword puzzle generation, established that combining forward checking with intelligent variable ordering consistently outperformed either technique in isolation. The techniques introduced in this paper form the direct basis for the solver implemented in the present project.

Bessiere [1] provided a comprehensive survey of constraint propagation techniques in the *Handbook of Constraint Programming*, tracing the development of consistency algorithms from Mackworth's original AC-3 through increasingly efficient successors such as AC-4 and AC-2001. Bessiere's [1] survey formalized the hierarchy of local consistency properties and demonstrated that the choice of propagation level represents a fundamental trade-off in CSP solving: stronger propagation reduces the search space more aggressively but incurs greater per-node computational cost. This trade-off is directly relevant to the present project, where forward checking (a relatively lightweight form of propagation) is compared against approaches that rely solely on heuristic variable ordering.

More recently, Simonis [6] applied constraint programming techniques directly to Sudoku in his paper *Sudoku as a Constraint Problem*, presented at the CP 2005 Workshop on Modelling and Reformulating Constraint Satisfaction Problems. Simonis modelled Sudoku using all different constraints for each row, column, and 3×3 block, and compared the effectiveness of different propagation schemes in solving puzzles without any backtracking search. His analysis showed that the propagation strength of the constraint model directly determines which puzzles can be solved by propagation alone, and he proposed an objective difficulty metric based on the level of propagation required. Simonis's [6] work is particularly relevant to this project because it demonstrates that Sudoku serves as an effective benchmark for comparing constraint satisfaction techniques of varying sophistication, from simple forward checking to full arc consistency and beyond.

This project investigates how different AI techniques affect the efficiency of solving Sudoku puzzles, applying the principles established in this prior work. Specifically, it compares baseline backtracking search with the Minimum Remaining Values (MRV) heuristic and forward checking, both individually and in combination. The goal is to evaluate how these techniques impact the performance of a Sudoku solver, using empirical metrics such as recursive calls, backtracking steps, and runtime.

Methods

Problem Formulation

The Sudoku puzzle is modelled as a constraint satisfaction problem [6]. Each of the 81 cells in the 9×9 grid represents a variable, and the domain of each variable consists of the digits 1 through 9. The constraints require that each digit appears exactly once in every row, exactly once in every column, and exactly once in every 3×3 subgrid. Pre-filled cells have their domains reduced to a single value. The solver's task is to find an assignment for all empty cells that satisfies every constraint.

The implementation represents the board as a two-dimensional list of integers, where 0 indicates an empty cell. A validity-checking function determines whether placing a given digit in a given cell violates any row, column, or subgrid constraint. A domain function computes the set of valid digits for any empty cell by testing each digit from 1 to 9 against the constraints.

Baseline Backtracking

The baseline solver uses standard backtracking search [3]. At each step, the algorithm selects the first unassigned variable encountered in a left-to-right, top-to-bottom scan of the grid and tries all digits from 1 through 9 in order. If a digit satisfies all constraints, the algorithm assigns it and proceeds recursively to the next empty cell. If no valid digit can be found for the current cell, the algorithm undoes the most recent assignment and returns to the previous cell to try a different value. This approach is guaranteed to find a solution if one exists but is computationally expensive because it lacks any strategy for choosing which variable to assign or which values to try first.

Minimum Remaining Values (MRV)

The Minimum Remaining Values heuristic improves variable selection by choosing the empty cell with the fewest remaining valid values in its domain, rather than simply selecting the first empty cell [2, 5]. This follows the "fail-first" principle: by prioritizing the most constrained cell, the algorithm is more likely to encounter a conflict early and backtrack before exploring a large subtree. When multiple cells share the same smallest domain size, ties are broken by scan order (top-left to bottom-right). Unlike the baseline solver, which iterates over all digits 1 through 9, the MRV solver iterates only over the cell's computed domain of valid values.

Forward Checking

Forward checking is a constraint propagation technique that operates as a lookahead after each assignment [2]. When a value is assigned to a cell, that value is removed from the domains of all neighbouring cells that share a row, column, or subgrid constraint. If any neighbouring cell is left with an empty domain, the algorithm immediately backtracks without recursing further. This prevents the solver from exploring branches that are guaranteed to fail, pruning the search tree earlier than standard backtracking. Forward checking can be understood as a limited form of arc consistency enforcement applied only at the point of assignment, rather than the full arc consistency maintained by algorithms such as AC-3 [1, 4].

In the implementation, a domains dictionary maps each cell to its set of remaining possible values. When the solver assigns a value, it creates a deep copy of the domains dictionary and applies the forward checking updates to the copy. This ensures that backtracking correctly restores the previous domain state. The forward checking solver uses the first-empty-cell variable selection strategy, the same as baseline backtracking, and iterates over domain values in sorted order for deterministic behaviour.

Combined Approach: MRV + Forward Checking

The final method combines MRV with forward checking. MRV determines which variable to assign next by consulting the maintained domains dictionary to find the cell with the fewest remaining values, while forward checking prunes the domains of neighbouring cells after each assignment. This combination aims to reduce both the depth of the search tree, by selecting the most constrained variable first, and the breadth of the search space, by eliminating invalid values from neighbouring domains before they are tried.

Test Data

Experiments were conducted on a set of nine Sudoku puzzles of varying difficulty, sourced from publicly available online puzzle collections. Each puzzle is a standard 9×9 grid with a unique solution. The puzzles range from having relatively few empty cells to having many, providing variation in the size of the search space.

Evaluation Metrics

To evaluate performance, three metrics were recorded for each solver on each puzzle. The first metric is the total number of recursive calls, which measures how many times the solver function was invoked during search. The second is the number of backtracking steps, defined as the number of times the algorithm exhausted all options for a cell and had to return to a previous assignment. The third is the total wall-clock runtime, measured using Python's time.perf_counter for high-resolution timing. All experiments were implemented in Python using only the standard library, with no external packages.

Results

The average performance of the four solving strategies across all nine puzzles is summarized in Table 1.

Table 1: Average performance metrics across nine puzzles.

Method	Avg. Calls	Avg. Backtracks	Avg. Time (s)
Backtracking	14,555.0	14,520.6	0.0770
MRV	89.9	55.4	0.0169
Forward Checking	3,965.8	3,931.3	0.1163
MRV + Forward Checking	85.7	51.2	0.0040

The baseline backtracking solver performed the worst across all metrics, requiring an average of over 14,000 recursive calls and nearly as many backtracking steps. This confirms that naive search without any heuristic guidance explores a very large portion of the search space.

The MRV heuristic dramatically improved performance. The average number of recursive calls dropped from 14,555 to approximately 90, a reduction of more than two orders of magnitude. Backtracking steps were similarly reduced from over 14,500 to about 55. This demonstrates that selecting the most constrained variable effectively focuses the search on cells where conflicts are most likely, enabling the solver to find solutions with minimal wasted exploration.

Forward checking also improved performance compared to baseline backtracking, reducing average recursive calls from 14,555 to approximately 3,966. However, it was notably less effective than MRV alone. Interestingly, the average runtime for forward checking (0.116 seconds) was higher than baseline backtracking (0.077 seconds), despite requiring fewer recursive calls. This overhead is attributable to the cost of copying and updating the domains dictionary at each step, which adds computational work per node even though fewer nodes are explored overall.

The best performance was achieved by combining MRV with forward checking. This approach produced the lowest average recursive calls (85.7), the fewest backtracking steps (51.2), and by far the fastest runtime (0.004 seconds). The combined method was approximately 19 times faster than baseline backtracking and 4 times faster than MRV alone, demonstrating that the two techniques complement each other: MRV focuses the search on the most constrained cells, while forward checking ensures that domains are kept as small as possible.

To illustrate the variation across puzzles, Table 2 presents per-puzzle recursive call counts for all four methods. The data reveals that performance differences are highly puzzle-dependent. For well-constrained puzzles (such as Puzzles 3 and 9), all methods converge quickly with similar call counts. For harder puzzles (such as Puzzle 2), baseline backtracking required over 56,000 recursive calls, while MRV + forward checking required only 254, a difference of more than two orders of magnitude.

Table 2: Recursive calls per puzzle for each solving method.

Puzzle	Backtracking	MRV	Fwd Checking	MRV + Fwd Check
1	4,209	52	334	52
2	56,663	271	16,218	254
3	55	46	46	46
4	4,607	14	520	13
5	726	3	131	2
6	64	23	23	22
7	2,932	52	213	52
8	61,684	302	18,161	284
9	55	46	46	46

As a sample of the solver's output, the following shows Puzzle 1 before and after being solved by the MRV + forward checking method. The solver completed the puzzle in 52 recursive calls with zero backtracking steps.

Input:

```
5 3 .  | . 7 .  | . . .
6 . .  | 1 9 5  | . . .
. 9 8  | . . .  | . 6 .
---------------------
8 . .  | . 6 .  | . . 3
4 . .  | 8 . 3  | . . 1
7 . .  | . 2 .  | . . 6
---------------------
. 6 .  | . . .  | 2 8 .
. . .  | 4 1 9  | . . 5
. . .  | . 8 .  | . 7 9
```

Solved:

```
5 3 4  | 6 7 8  | 9 1 2
6 7 2  | 1 9 5  | 3 4 8
1 9 8  | 3 4 2  | 5 6 7
---------------------
8 5 9  | 7 6 1  | 4 2 3
4 2 6  | 8 5 3  | 7 9 1
7 1 3  | 9 2 4  | 8 5 6
---------------------
9 6 1  | 5 3 7  | 2 8 4
2 8 7  | 4 1 9  | 6 3 5
3 4 5  | 2 8 6  | 1 7 9
```

Conclusion

This project demonstrates how different AI techniques influence the efficiency of solving Sudoku as a constraint satisfaction problem. The results highlight several key findings that align with established theory in the CSP literature.

First, naive backtracking is highly inefficient due to the size of the search space. Without any guidance for variable selection or domain pruning, the algorithm explores many branches that could have been avoided, resulting in thousands of unnecessary recursive calls even on moderately difficult puzzles.

Second, the MRV heuristic provides the single largest improvement among the techniques tested. By prioritizing the most constrained variable at each step, MRV reduces the branching factor and detects dead ends early, consistent with the fail-first principle described by Haralick and Elliott [2]. The dramatic reduction in recursive calls, from an average of 14,555 to 90, demonstrates that intelligent variable ordering is more impactful than constraint propagation alone for this problem.

Third, forward checking improves pruning compared to baseline backtracking but is less effective when used in isolation. The per-node overhead of maintaining and copying domain dictionaries partially offsets the benefit of reduced exploration, as reflected in the slightly higher average runtime compared to baseline backtracking. However, when combined with MRV, forward checking provides an additional speedup by keeping domains tightly pruned, enabling MRV to make better-informed variable selections.

The combined MRV + forward checking approach produced the best results across all metrics and was the only method to achieve sub-millisecond average runtime. This finding is consistent with the conclusions of Haralick and Elliott [2], who demonstrated that combining

lookahead techniques with heuristic ordering consistently outperforms either approach alone. The forward checking technique used in this project represents one level of constraint propagation; as Mackworth [4] formalized, stronger propagation such as arc consistency (AC-3) can eliminate more inconsistent values before search begins, potentially reducing the search tree even further. As Bessiere [1] detailed in his comprehensive survey, the hierarchy of consistency properties offers a continuum of trade-offs between propagation strength and computational cost, suggesting that intermediate approaches such as maintaining arc consistency (MAC) during search could yield further improvements for Sudoku. Simonis [6] demonstrated that varying levels of propagation strength determine which Sudoku puzzles can be solved without any backtracking at all, reinforcing this point. Following the general framework presented by Russell and Norvig [5], further improvements could also include the Least Constraining Value (LCV) heuristic for value ordering.

One limitation of this work is the relatively small set of nine test puzzles. A larger and more diverse dataset, including puzzles specifically designed to be difficult for backtracking solvers, would provide a more robust evaluation. Additionally, the current implementation creates a full deep copy of the domains dictionary at each forward checking step, which introduces overhead that could be reduced using a trail-based undo mechanism. Future work could address both of these limitations while also exploring the additional heuristics mentioned above.

Despite these limitations, the project successfully demonstrates how constraint satisfaction techniques and heuristics can dramatically improve search efficiency in a well-structured combinatorial problem.

References

[1] Bessiere, C. (2006). Constraint propagation. In F. Rossi, P. van Beek, & T. Walsh (Eds.), *Handbook of constraint programming* (pp. 29–83). Elsevier.

[2] Haralick, R. M., & Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, *14*(3), 263–313.

[3] Kumar, V. (1992). Algorithms for constraint-satisfaction problems: A survey. *AI Magazine*, *13*(1), 32–44.

[4] Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence*, *8*(1), 99–118.

[5] Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

[6] Simonis, H. (2005). Sudoku as a constraint problem. In B. Hnich, P. Prosser, & B. Smith (Eds.), *Proceedings of the 4th International Workshop on Modelling and Reformulating Constraint Satisfaction Problems* (pp. 13–27).